

Sqillify

From job ads to skills trends and candidate matches

Python · NiFi · Kafka · Spark · MySQL · dbt · Airflow · Tableau · Slack

PATRICIA MOUTINHO

Batch July 2026

1. The problem

Too many jobs ads. Too much noise. Too many emails, notifications and job recommendations that after reading the job description, were not a match.

- Which jobs actually match my skills?
- Which skills are employers looking for?



JOB ADS

Different titles
Different wording
HTML descriptions
Many skills

SQILLIFY

Collect
Standardise
Transform
Match

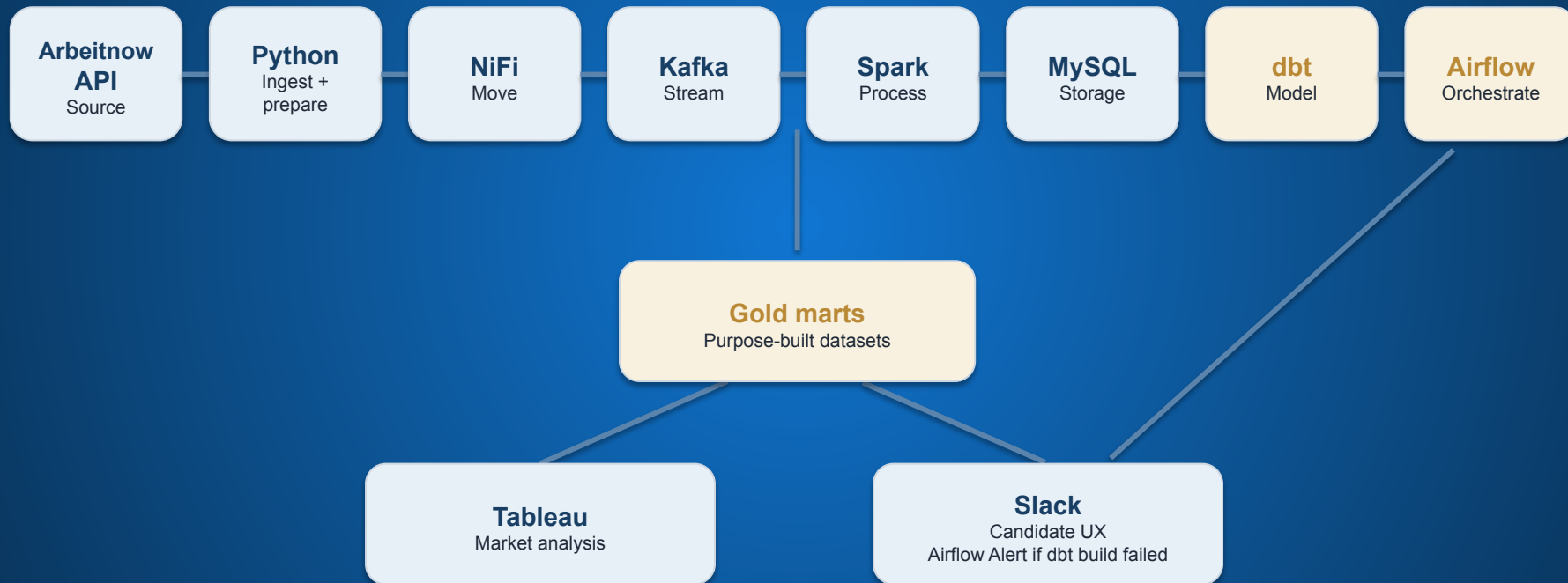
USEFUL OUTPUT

Market insights
Skill demand
Candidate matches
Job links

Goal: turn a stream of job advertisements into data that can answer questions and, above all, support a candidate looking for a job.

2. Sqillify Pipeline Overview

Each tool has one clear job in the journey.



3. Ingest and prepare

Keep the source, then create a clean standard format.



Bronze = preserve.

Silver = clean and standardise.

The image shows a VS Code editor interface. On the left is the Explorer sidebar showing a project structure with folders like `sqlify`, `src`, and files like `collect_jobs.py`. The main editor area displays the `collect_jobs.py` file, which is a Python script that uses `requests` to fetch data from the `Arbeitsagentur` API and saves it to a JSON file. The script includes comments and uses `Path` from `pathlib` for file handling. The terminal at the bottom shows the command to run the script, the status code (200), the number of jobs received (175), and a detailed JSON output for the first job, including title, company, location, and description. The raw JSON data is also shown at the bottom of the terminal.

```
src > collect_jobs.py > ...
1 import json
2 from pathlib import Path
3 import requests
4
5 API_URL = "https://www.arbeitsagentur.com/api/job-board-api"
6
7 PROJECT_ROOT = Path(__file__).resolve().parent.parent
8 OUTPUT_FILE = PROJECT_ROOT / "data" / "bronze" / "raw_jobs.json"
9
10 response = requests.get(API_URL, timeout=30)
11 response.raise_for_status()
12
13 api_result = response.json()
14 jobs = api_result["data"]
15
16 # save the original jobs, no transformation
17 with OUTPUT_FILE.open("w", encoding="utf-8") as file:
18     json.dump(jobs, file, ensure_ascii=False, indent=2)
19
20 print("Status code:", response.status_code)
21 print("Number of jobs received:", len(jobs))
22
23 first_job = jobs[0]
24
25 print("\nFirst job:")
26 print("Title:", first_job["title"])
```

Terminal Output:

```
source /Users/Bootcamp/Desktop/Labs/sqlify/.venv/bin/activate
(.venv) → sqlify source /Users/Bootcamp/Desktop/Labs/sqlify/.venv/bin/activate
(.venv) → sqlify git:(main) x python src/collect_jobs.py
Status code: 200
Number of jobs received: 175

First job:
Title: Account Executive - New Business
Company: Fiskaly
Location: Berlin, Berlin
Remote: False
Job types: []
Tags: ['Sales']
Created at: 1788468920
URL: https://www.arbeitsagentur.com/jobs/companies/fiskaly/account-executive-new-business-berlin-berlin-353654
Description preview: <h2 data-start="1186" data-end="1366">Job Details</h2>
a-end="1366">Job Details</h2>
<ul>
<li><strong>
Raw jobs saved to: /Users/Bootcamp/Desktop/Labs/sqlify/data/bronze/raw_jobs.json
(.venv) → sqlify git:(main) x
```

Data Source: **API ArbeitNow**

Python `collect_jobs.py` to collect and save raw data in **Bronze** layer.

Explorer

- sqllify
 - .venv
 - airflow
 - data
 - bronze
 - raw_jobs.json
 - checkpoints
 - silver
 - standard_jobs.json
 - dbt_packages
 - docs
 - kafka-data
 - logs
 - sqllify_dbt
 - sql
 - src
 - __pycache__
 - collect_jobs.py
 - database.py
 - dbt_runner.py
 - job_results.py
 - matching.py
 - prepare_jobs.py
 - slack_bot.py
 - slack_options.py
 - slack_views.py
 - stream_jobs.py
 - tests
 - .env
 - .gitignore
 - jobs_by_experience.csv
 - most_wanted_skills.csv
 - remote_or_not.csv
 - requirements.txt
 - skills.json
 - sqllify_schema.sql

data > silver > standard_jobs.json > {} 0

This document contains many ambiguous unicode characters [Disable Ambiguous Highlight](#)

```
1
2 {
3   "job_id": "account-executive-new-business-berlin-berlin-353654",
4   "title": "Account Executive - New Business",
5   "company": "Fiskaly",
6   "location": "Berlin, Berlin",
7   "remote": false,
8   "job_types": [],
9   "tags": [
10     "Sales"
11   ],
12   "description": "Job Details Role: Account Executive - New Business Location: Remote",
13   "published_at": "2026-09-03T20:55:20Z",
14   "job_url": "https://www.arbeitnow.com/jobs/companies/fiskaly/account-executive-new-",
15   "source": "arbeitnow"
16 },
17 {
18   "job_id": "senior-functional-safety-autosar-classic-software-engineer-freelancer-mu",
19   "title": "Senior Functional Safety & Autosar Classic Software Engineer Freelancer (",
20   "company": "Paradoxcatsgmbh",
21   "location": "Ingolstadt, Bavaria; München, Bavaria",
22   "remote": false,
23   "job_types": [],
24   "tags": [
25     "Software Engineering"
26   ],
27   "description": "Our company Paradox Cat GmbH is a specialist software development p",
28   "published_at": "2026-09-03T20:55:16Z",
29   "job_url": "https://www.arbeitnow.com/jobs/companies/paradoxcatsgmbh/senior-function",
30   "source": "arbeitnow"
31 },
32 {
33   "job_id": "senior-embedded-platform-linux-engineer-freelancer-munich-munich-bavari",
34   "title": "Senior Embedded Platform & Linux Engineer Freelancer (F/M/D) - Munich",
35   "company": "Paradoxcatsgmbh",
36   "location": "München, Bavaria",
37   "remote": false,
38   "job_types": [],
39   "tags": [
```

Ports Query Results Lineage Query Results Compare (Beta) Catalog (Beta) Terminal

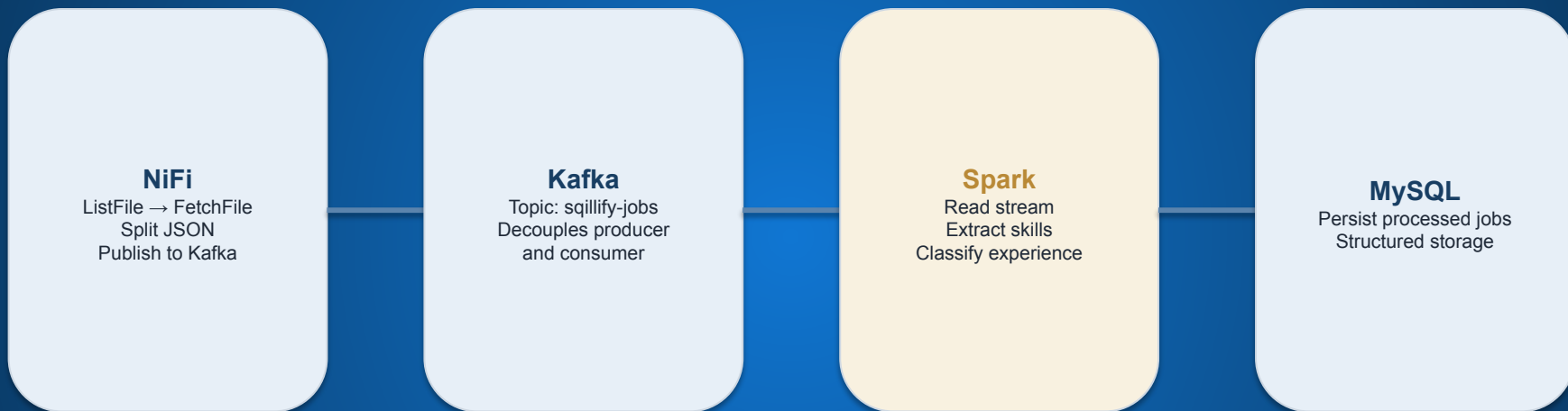
```
(.venv) → sqllify git:(main) x python src/prepare_jobs.py
Prepared 175 jobs.

Silver data saved to: /Users/Bootcamp/Desktop/Labs/sqllify/data/silver/standard_jobs.json
(.venv) → sqllify git:(main) x
```

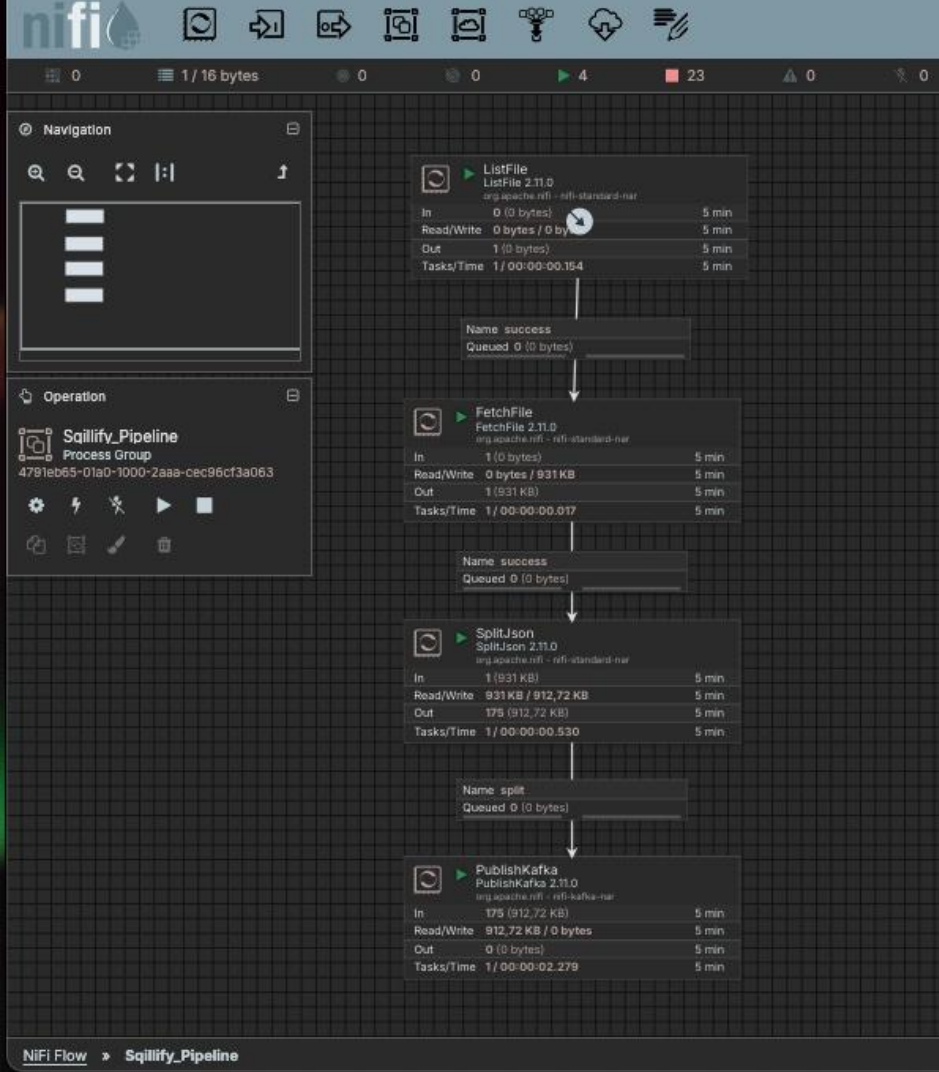
Python preparation_jobs.py cleans HTML encoded, standardizes fields and stores data in **Silver** layer.

4. Move, stream and process

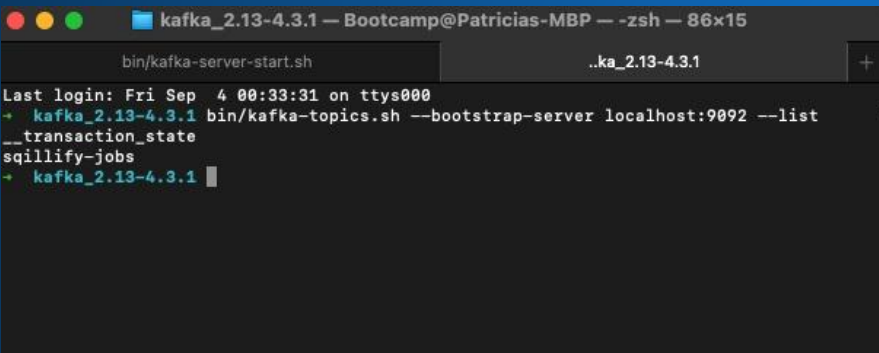
The Silver dataset becomes an **event stream** that Spark can transform.



NiFi sends the job data to Kafka. Kafka holds the messages until Spark picks them up. Spark processes the data and saves the results in MySQL.

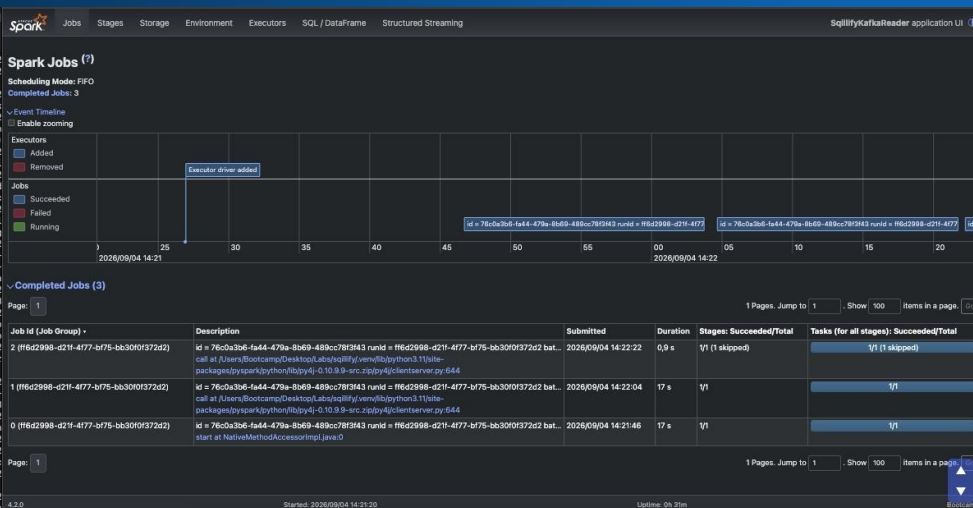


Apache NiFi reads the Silver job data, splits the JSON records and publishes them to Kafka.



```
kafka_2.13-4.3.1 — Bootcamp@Patricias-MBP — zsh — 86x15
bin/kafka-server-start.sh      ..ka_2.13-4.3.1
Last login: Fri Sep  4 00:33:31 on ttys000
➔ kafka_2.13-4.3.1 bin/kafka-topics.sh --bootstrap-server localhost:9092 --list
__transaction_state
sqillify-jobs
➔ kafka_2.13-4.3.1
```

Apache Kafka is responsible for streaming/ messaging. It transports individual job records through the sqillify-jobs topic.



Apache Spark consumes Kafka data, extracts skills, classifies experience levels and prepares job records.

Local instance 3306 - Warning - not supported

Administration Schemas SQL File 4* SQL File 5*

SCHEMAS

Filter objects

- airflow_dbt_lab
- dbt_lab
- kafka_lab
- quickcart
- sqllify
- Tables
- Views
- Stored Procedures
- Functions
- sys

```
1 USE sqllify;
2
3 SELECT title,company, experience_level,ad_skills_found
4 FROM jobs
5 WHERE experience_level IN ('junior')
6 LIMIT 10;
7
8
```

100% 1/7

Result Grid Filter Rows: Search Export: Fetch rows:

title	company	experience_level	ad_skills_found
CAD & PLM Administration Intern	The Exploration Company	junior	["c#", "linux"]
Finance and Accounting Intern	Andercore	junior	[]
Internship - Machine Learning Research...	Perplexity	junior	[]
Jr. Product Manager	Doonalls	junior	[]
(Junior) Accounting Manager*in I Scal...	Matera GmbH	junior	[]
Junior Customer Programs Manager DA...	SoSafe	junior	[]
Junior Customer Programs Manager DA...	SoSafe	junior	[]
Junior Fachinformatiker/-in (m/w/d) – Vo...	Cordes Consulting GmbH	junior	[]
Junior Google Performance & Analytics...	eFLY Marketplace Services GmbH	junior	[]
(Junior) Grafikdesign / Mediengestaltun...	premano GmbH & Co. KG	junior	[]

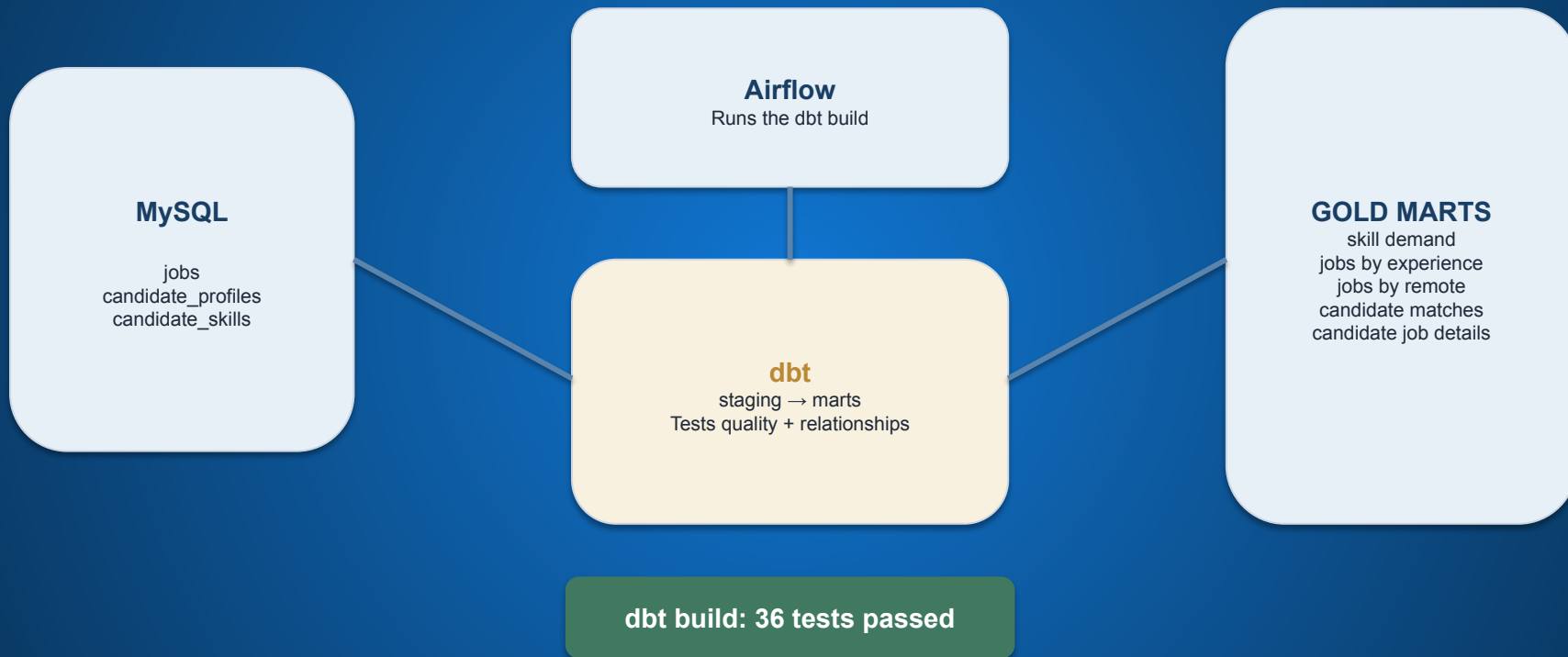
jobs 18 Read Only

Action Output

7/1 16:00:20 SELECT job_id, title, co... 10 rows returned 0.076 sec / 11.000000

MySQL stores processed jobs, candidate profiles and candidate skills.

5. Model and orchestrate



6. dbt marts

Ready-to-use dataset built for Tableau and Slack.

`mart_skill_demand`

Which skills appear most?

`mart_jobs_by_experience`

How is demand split by level?

`mart_jobs_by_remote`

How much work is remote?

`mart_candidate_job_matches`

Which jobs match a candidate?

`mart_candidate_job_details`

What should Slack show?

The screenshot displays the SQuillify IDE interface. On the left, the Explorer pane shows a project structure with folders like 'venv', 'airflow', 'data', 'dbt_packages', 'docs', 'kafka-data', 'logs', 'sqlify_dbt', 'analyses', 'dbt_packages', 'logs', 'macros', 'models', and 'marts'. The 'marts' folder is expanded, showing files such as 'mart_candidate_job_details.sql', 'mart_candidate_job_matches.sql', 'mart_jobs_by_experience.sql' (which is selected), 'mart_jobs_by_remote.sql', 'mart_skill_demand.sql', 'staging', 'seeds', 'snapshots', 'target', 'tests', '.gitignore', 'dbt_project.yml', and 'README.md'. Below this, a file explorer shows 'jobs_by_experience.csv', 'most_wanted_skills.csv', 'remote_or_not.csv', 'requirements.txt', 'skills.json', and 'sqlify_schema.sql'.

The main editor displays a SQL query in 'mart_jobs_by_experience.sql':

```
1 {{ config(materialized='table') }}
2
3 select experience_level, count(distinct job_id) as job_count
4 from {{ ref('stg_jobs') }}
5 group by experience_level
```

The bottom pane shows the 'Terminal' output, which includes the command to run dbt tests and the resulting output:

```
(.venv) → sqlify_dbt git:(main) × python -m dbt.cli.main build
23:51:19 29 of 36 PASS not_null_mart_candidate_job_details_candidate_id ..... [PASS in 0.09s]
23:51:19 30 of 36 START test not_null_mart_candidate_job_details_company ..... [RUN]
23:51:19 30 of 36 PASS not_null_mart_candidate_job_details_company ..... [PASS in 0.06s]
23:51:19 31 of 36 START test not_null_mart_candidate_job_details_job_id ..... [RUN]
23:51:19 31 of 36 PASS not_null_mart_candidate_job_details_job_id ..... [PASS in 0.10s]
23:51:19 32 of 36 START test not_null_mart_candidate_job_details_matched_skill_count ..... [RUN]
23:51:19 32 of 36 PASS not_null_mart_candidate_job_details_matched_skill_count ..... [PASS in 0.04s]
23:51:19 33 of 36 START test not_null_mart_candidate_job_details_remote_status ..... [RUN]
23:51:19 33 of 36 PASS not_null_mart_candidate_job_details_remote_status ..... [PASS in 0.05s]
23:51:19 34 of 36 START test not_null_mart_candidate_job_details_title ..... [RUN]
23:51:19 34 of 36 PASS not_null_mart_candidate_job_details_title ..... [PASS in 0.06s]
23:51:19 35 of 36 START test relationships_mart_candidate_job_details_candidate_id_candidate_id_source_sqlify_candidate_profiles ..... [RUN]
23:51:19 35 of 36 PASS relationships_mart_candidate_job_details_candidate_id_candidate_id_source_sqlify_candidate_profiles ..... [PASS in 0.05s]
23:51:19 36 of 36 START test relationships_mart_candidate_job_details_job_id_job_id_ref_stg_jobs ..... [RUN]
23:51:19 36 of 36 PASS relationships_mart_candidate_job_details_job_id_job_id_ref_stg_jobs ..... [PASS in 0.05s]
23:51:19 Finished running 1 view model, 30 tests, 5 table models in 0 hours 0 minutes and 5.56 seconds (5.56s).
23:51:19 Completed successfully
23:51:19 Done. PASS=36 WARN=0 ERROR=0 SKIP=0 TOTAL=36
(.venv) → sqlify_dbt git:(main) ×
```

dbt transforms database data into tested staging models and Gold marts.

The screenshot shows the Apache Airflow web interface. At the top, a summary bar for the DAG run 'manual__2026-09-03T11:48:49.636797+00:00' indicates a 'Success' status. Below this, a table lists the tasks in the DAG. All tasks are marked as 'Success'.

Task ID	Map Index	State	Start Date	Try Number	Operator	Duration
dbt_build		Success	2026-09-03 13:49:02	1	BashOperator	00:00:28.291
wait_for_fresh_mysql		Success	2026-09-03 13:48:56	1	SqlSensor	00:00:05.249
prepare_jobs		Success	2026-09-03 13:48:53	1	BashOperator	00:00:02.420
collect_jobs		Success	2026-09-03 13:48:51	1	BashOperator	00:00:01.933

Apache Airflow orchestrates the collection and preparation stages, waits for fresh processed data to arrive in MySQL, and then triggers the dbt build.

The screenshot shows a Slack interface with a channel named '# all-sqillifyairflowalerts'. A message from the 'Sqillify' app is visible, titled 'Sqillify Airflow failure'. The message content provides details about the failed DAG run, including the DAG name, task name, run ID, and a log link.

Sqillify Airflow failure
 DAG: sqillify_dbt_build
 Task: dbt_build
 Run: manual__2026-09-02T19:44:54.958001+00:00
 Log: http://localhost:8080/dags/sqillify_dbt_build/runs/manual__2026-09-02T19%3A44%3A54.958001%2B00%3A00/tasks/dbt_build?try_number=3

If dbt does not build data, the task fails and Airflow will notify me in Slack.

7. Two consumers, one prepared data layer

The Gold marts support analysis and an interactive candidate experience.

TABLEAU

Skill demand
Jobs by experience
Remote status
Interactive dashboard

← **GOLD** →

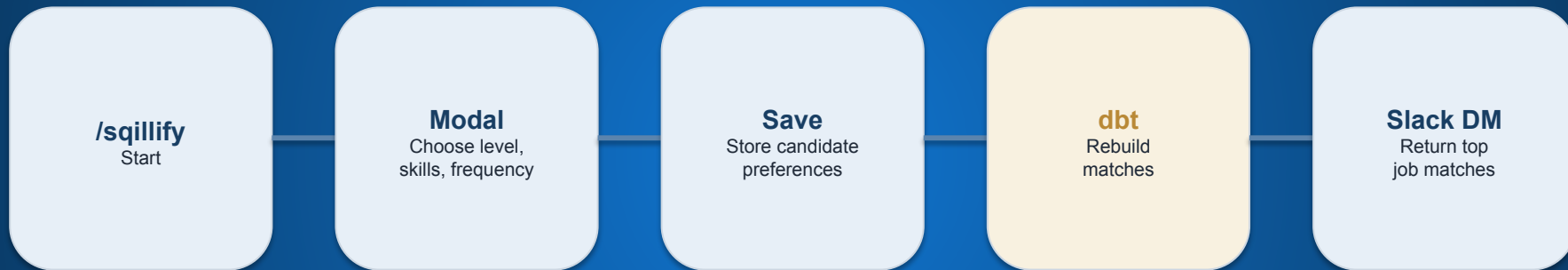
SLACK

/sqillify command

Experience selection
Skill selection
Match frequency
Job recommendations

8. Candidate flow in Slack

A lightweight interface over the matching pipeline.



Prototype limitations:

- The candidate can already save a preferred match frequency. In the current prototype, the weekly preference is stored but not yet connected to a scheduled Airflow workflow. The matching itself is currently triggered immediately.
- The current Slack implementation uses `candidate_id = 1`. A production version would map each Slack user to an individual candidate profile.

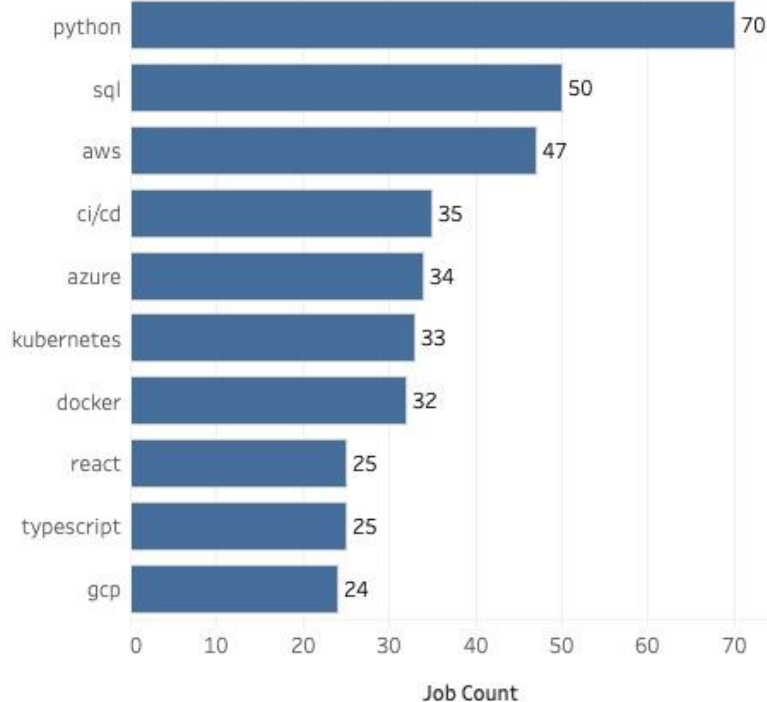
Device Preview

Automatic

- Top Skills in Demand
- Jobs by Level of Experience
- Remote status

- Horizontal Container
- Vertical Container
- Text
- Extension
- Image
- Blank
- Workflow
- Web Page

Floating

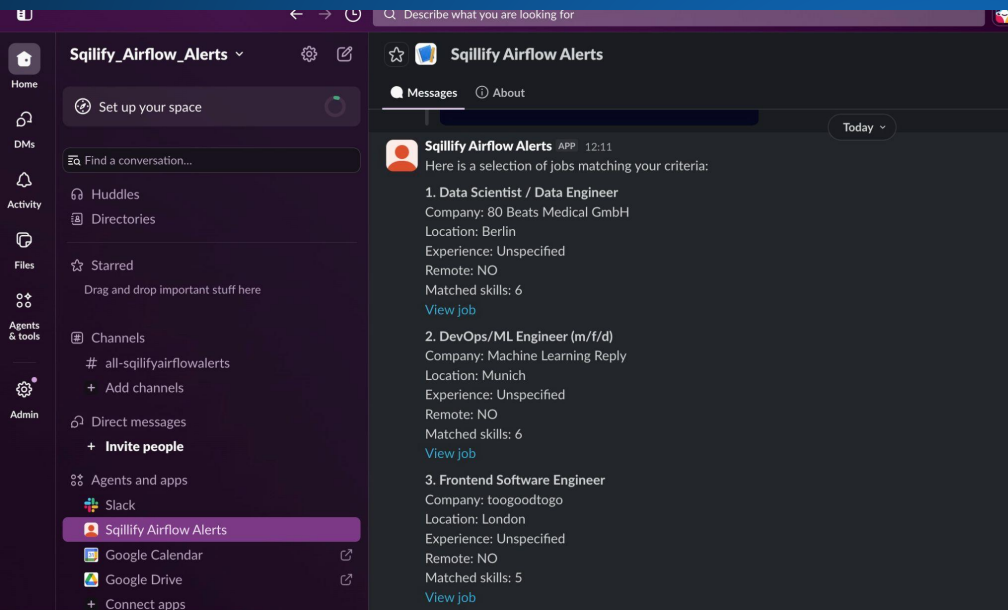
☐ Show dashboard title

Frequency	Count
Daily	278
Weekly	197
Monthly	47
Never	19

- junior
- senior
- student
- unspecified

☐ NO
☐ YES

Remote Status	Job Count
NO	~500
YES	~10



Slack serves as **the user interface** via command **/sqillify** which enables a modal drop down menu for the candidate select her/his preferences. Then it returns a top 10 matched jobs via slack message. (see image)

So instead of going through all 175 job descriptions manually, the candidate can now ask Sqillify for the jobs that best match their selected skills!

10. Live demo

Now follow one job-ad journey from data to decision.

Step 1: Collect jobs from the API ArbeitNow

Step 2: Prepare jobs → Bronze becomes Silver.

Step 3: NiFi processors

Step 4: Kafka consumer

Step 5 : Spark

Step 6: MySQL

Step 7: dbt marts and dbt build via Airflow

Step 8: Airflow running on Docker container

Step 9: Tableau for job market analyse

Step 10 : Slack App integration

Thank you!